Problemario

Eliminación Gaussiana, Gauss-Jordan, Gauss-Seidel y Jacobi

Métodos Numéricos

ITESA

Dr. Efrén Rolando Romero

Fecha: 13 de marzo de 2026

Índice

Eliminación Gaussiana

Algoritmo

1. Escribir el sistema de ecuaciones lineales.
2. Formar la matriz aumentada del sistema.
3. Elegir el primer elemento de la diagonal principal como pivote.
4. Utilizar el pivote para eliminar los elementos debajo de él mediante operaciones entre filas.
5. Repetir el proceso de eliminación para cada columna hasta obtener ceros debajo de la diagonal principal.
6. Obtener una matriz triangular superior.
7. Calcular el valor de la última variable usando la última ecuación.
8. Sustituir ese valor en la ecuación anterior para encontrar la siguiente variable.
9. Continuar la sustitución hacia atrás hasta encontrar todas las variables.
10. Mostrar los valores obtenidos como la solución del sistema.

Código

```
using System;

namespace GaussianElimination
{
   class Program
   {
      static void Main(string[] args)
      {
         Console.Write("Ingrese el tamaño del sistema (n): ");
         int n = int.Parse(Console.ReadLine());

         // Leer matriz aumentada [A|b]
         Console.WriteLine($"Ingrese la matriz aumentada ({n} x {n + 1}):");
         double[,] matrix = new double[n, n + 1];

         for (int i = 0; i < n; i++)
         {
            Console.Write($"Fila {i + 1}: ");
            string[] values = Console.ReadLine().Split();

            if (values.Length != n + 1)
```

```
    {
        Console.WriteLine($"Error: Cada fila debe tener exactamente {n + 1} elementos.");
        return;
    }

    for (int j = 0; j < n + 1; j++)
    {
        matrix[i, j] = double.Parse(values[j]);
    }
}

// Eliminación hacia adelante con pivoteo parcial
for (int i = 0; i < n; i++)
{
    // Buscar fila pivote
    int maxRow = i;
    for (int k = i + 1; k < n; k++)
    {
        if (Math.Abs(matrix[k, i]) > Math.Abs(matrix[maxRow, i]))
        {
            maxRow = k;
        }
    }

    // Intercambiar filas
    if (maxRow != i)
    {
        for (int j = 0; j < n + 1; j++)
        {
            double temp = matrix[i, j];
            matrix[i, j] = matrix[maxRow, j];
            matrix[maxRow, j] = temp;
        }
    }

    // Verificar si la matriz es singular
    if (Math.Abs(matrix[i, i]) < 1e-10)
    {
        Console.WriteLine("Error: La matriz es singular o casi singular.");
        return;
    }

    // Eliminar columna
    for (int k = i + 1; k < n; k++)
    {
        double factor = matrix[k, i] / matrix[i, i];
        for (int j = i; j < n + 1; j++)
        {
```

```
                matrix[k, j] -= factor * matrix[i, j];
            }
        }
    }

    // Sustitución hacia atrás
    double[] x = new double[n];
    for (int i = n - 1; i >= 0; i--)
    {
        x[i] = matrix[i, n];
        for (int j = i + 1; j < n; j++)
        {
            x[i] -= matrix[i, j] * x[j];
        }
        x[i] /= matrix[i, i];
    }

    // Mostrar solución
    Console.WriteLine("\nSolución:");
    for (int i = 0; i < n; i++)
    {
        Console.WriteLine($"x{i + 1} = {x[i]:F6}");
    }

    Console.WriteLine("\nPresione cualquier tecla para salir...");
    Console.ReadKey();
  }
 }
}
```

Ejercicio 1

$$\left[\begin{array}{cc|c} 2 & 1 & 5 \\ 4 & -6 & -2 \end{array}\right]$$

```
Ingrese el tamaño del sistema (n): 2
Ingrese la matriz aumentada (2 x 3):
Fila 1: 2 1 5
Fila 2: 4 -6 -2

Solución:
x1 = 1.750000
x2 = 1.500000
```

Ejercicio 2:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array}\right]$$

```
Ingrese el tamaño del sistema (n): 3
Ingrese la matriz aumentada (3 x 4):
Fila 1: 2 1 -1 8
Fila 2: -3 -3 2 -11
Fila 3: -2 1 2 -3

Solución:
x1 = 5.600000
x2 = 0.600000
x3 = 3.800000
```

Ejercicio 3:

$$\left[\begin{array}{cccc|c} 1 & 2 & -1 & 3 & 9 \\ 2 & -1 & 1 & 1 & 8 \\ 3 & 1 & -2 & 2 & 3 \\ 1 & 3 & 2 & -1 & 10 \end{array}\right]$$

```
Ingrese el tamaño del sistema (n): 4
Ingrese la matriz aumentada (4 x 5):
Fila 1: 1 2 -1 3 9
Fila 2: 2 -1 1 1 8
Fila 3: 3 1 -2 2 8
Fila 4: 1 3 2 -1 10

Solución:
x1 = 2.500000
x2 = 1.500000
x3 = 2.500000
x4 = 2.000000
```

Ejercicio 4:

$$\left[\begin{array}{cccccccccc|c} 10 & 2 & 3 & 4 & 5 & 1 & 0 & 2 & 3 & 1 & 31 \\ 1 & 11 & 2 & 3 & 4 & 2 & 1 & 0 & 1 & 2 & 27 \\ 2 & 1 & 12 & 2 & 3 & 1 & 2 & 1 & 0 & 1 & 25 \\ 3 & 2 & 1 & 13 & 2 & 0 & 1 & 2 & 1 & 1 & 26 \\ 1 & 3 & 2 & 1 & 14 & 2 & 1 & 0 & 2 & 1 & 27 \\ 2 & 1 & 3 & 2 & 1 & 15 & 2 & 1 & 0 & 1 & 28 \\ 1 & 2 & 1 & 3 & 2 & 1 & 16 & 2 & 1 & 0 & 29 \\ 2 & 1 & 2 & 1 & 3 & 2 & 1 & 17 & 2 & 1 & 32 \\ 1 & 2 & 1 & 2 & 1 & 3 & 2 & 1 & 18 & 2 & 33 \\ 2 & 1 & 2 & 1 & 2 & 1 & 3 & 2 & 1 & 19 & 34 \end{array}\right]$$

```
Ingrese el tamaño del sistema (n): 10
Ingrese la matriz aumentada (10 x 11):
Fila 1: 10 2 3 4 5 1 0 2 3 1 31
Fila 2: 1 11 2 3 4 2 1 0 1 2 27
Fila 3: 2 1 12 2 3 1 2 1 0 1 25
Fila 4: 3 2 1 13 2 0 1 2 1 1 26
Fila 5: 1 3 2 1 14 2 1 0 2 1 27
Fila 6: 2 1 3 2 1 15 2 1 0 1 28
Fila 7: 1 2 1 3 2 1 16 2 1 0 29
Fila 8: 2 1 2 1 3 2 1 17 2 1 32
Fila 9: 1 2 1 2 1 3 2 1 18 2 33
Fila 10: 2 1 2 1 2 1 3 2 1 19 34

Solución:
x1 = 1.000000
x2 = 1.000000
x3 = 1.000000
x4 = 1.000000
x5 = 1.000000
x6 = 1.000000
x7 = 1.000000
x8 = 1.000000
x9 = 1.000000
x10 = 1.000000
```

Ejercicio 5:

$$
\left[\begin{array}{cccccccccccccc|c}
1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 120 \\
2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 140 \\
3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 160 \\
4 & 5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 180 \\
5 & 6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 200 \\
6 & 7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 220 \\
7 & 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 240 \\
8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 260 \\
9 & 10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 280 \\
10 & 11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 300 \\
11 & 12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 & 320 \\
12 & 13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 & 26 & 340 \\
13 & 14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 & 26 & 27 & 360 \\
14 & 15 & 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 & 24 & 25 & 26 & 27 & 28 & 380 \\
2 & 4 & 6 & 8 & 10 & 12 & 14 & 16 & 18 & 20 & 22 & 24 & 26 & 28 & 30 & 240
\end{array}\right]
$$

```
Ingrese el tamaño del sistema (n): 15
Ingrese la matriz aumentada (15 x 16):
Fila 1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 120
Fila 2: 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 140
Fila 3: 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 160
Fila 4: 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 180
Fila 5: 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 200
Fila 6: 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 220
Fila 7: 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 240
Fila 8: 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 260
Fila 9: 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 280
Fila 10: 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 300
Fila 11: 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 320
Fila 12: 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 340
Fila 13: 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 360
Fila 14: 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 380
Fila 15: 2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 240
Error: La matriz es singular o casi singular.
```

Ejercicio 6:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	210
2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	230
3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	250
4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	270
5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	290
6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	310
7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	330
8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	350
9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	370
10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	390
11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	410
12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	430
13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	450
14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	470
15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	490
16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	510
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	530
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	550
19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	570
1	2	3	4	5	6	7	8	9	1	11	12	13	14	15	16	17	18	19	20	210

```
Ingrese el tamaño del sistema (n): 20
Ingrese la matriz aumentada (20 x 21):
Fila 1: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 210
Fila 2: 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 230
Fila 3: 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 250
Fila 4: 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 270
Fila 5: 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 290
Fila 6: 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 310
Fila 7: 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 330
Fila 8: 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 350
Fila 9: 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 370
Fila 10: 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 390
Fila 11: 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 410
Fila 12: 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 430
Fila 13: 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 450
Fila 14: 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 470
Fila 15: 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 490
Fila 16: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 510
Fila 17: 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 530
Fila 18: 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 550
Fila 19: 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 570
Fila 20: 1 2 3 4 5 6 7 8 9 14 11 12 13 14 15 16 17 18 19 20 210
Error: La matriz es singular o casi singular.
```

Método de Eliminación de Gauss-Jordan

Algoritmo

Para resolver un sistema de ecuaciones, seguimos estos pasos lógicos:

1. **Matriz Aumentada:** Escribe el sistema de ecuaciones en forma de matriz combinando la matriz de coeficientes A y el vector de resultados B.
2. **Pivoteo:** En la primera columna, selecciona el elemento de la diagonal principal (el pivote). Si es cero, intercambia la fila con una inferior que no tenga un cero en esa posición.
3. **Normalización:** Divide toda la fila del pivote por el valor del propio pivote para convertirlo en **1**.
4. **Eliminación:** Suma o resta múltiplos de la fila del pivote a **todas** las demás filas (tanto arriba como abajo) para que los demás elementos de esa columna se conviertan en **0**.
5. **Repetición:** Repite el proceso para la siguiente columna, desplazándote por la diagonal principal hasta llegar a la última fila.

Código:

```python
import numpy as np

def gauss_jordan_corto(A, B):
    # 1. Matriz Aumentada
    Ab = np.hstack([A.astype(float), B.reshape(-1, 1).astype(float)])
    n = len(B)

    for i in range(n):
        # 2. Pivoteo (Intercambio si el pivote es 0)
        if Ab[i, i] == 0:
            idx_max = np.argmax(np.abs(Ab[i:, i])) + i
            Ab[[i, idx_max]] = Ab[[idx_max, i]]

        # 3. Normalización (Fila del pivote / valor pivote)
        Ab[i] = Ab[i] / Ab[i, i]

        # 4. Eliminación (Hacer 0s arriba y abajo)
        for j in range(n):
            if i != j:
                Ab[j] -= Ab[j, i] * Ab[i]

        # 5. Repetición (Controlada por el ciclo for i)

    return Ab[:, -1] # Retorna la última columna (soluciones)

# Prueba rápida
A = np.array([[2, -1, 1], [1, 1, 2], [3, -1, -1]])
B = np.array([2, 9, -2])

print("Soluciones:", gauss_jordan_corto(A, B))
```

Ejercicio 1

Matriz de 2x2

Sistema de Ecuaciones:

1. $2x + 1y = 5$
2. $1x + 3y = 10$

```
def resolver_2x2_paso_a_paso(A, B):
    # PASO 1: Matriz Aumentada
    Ab = np.hstack([A.astype(float), B.reshape(-1, 1).astype(float)])
    n = len(B)
    print("--- PASO 1: Matriz Aumentada Inicial ---")
    print(Ab)
    for i in range(n):
        # PASO 2: Pivoteo
        if Ab[i, i] == 0:
            print(f"Pivote en [{i},{i}] es cero. Buscando intercambio...")
            max_row = np.argmax(np.abs(Ab[i:, i])) + i
            Ab[[i, max_row]] = Ab[[max_row, i]]
            print("Matriz después del intercambio de filas:")
            print(Ab)
        # PASO 3: Normalización
        pivote = Ab[i, i]
        Ab[i] = Ab[i] / pivote
        print(f"PASO 3: Fila {i+1} normalizada (Pivote = 1):")
        print(Ab)
        # PASO 4: Eliminación
        for j in range(n):
            if i != j:
                factor = Ab[j, i]
                Ab[j] = Ab[j] - factor * Ab[i]
                print(f"PASO 4: Eliminando elemento en [{j},{i}] usando factor
                {factor}:")
                print(Ab)
        # PASO 5: Repetición
    return Ab[:, -1]
# Sistema: 2x + 1y = 5  |  1x + 3y = 10
A_2x2 = np.array([[2, 1],
                  [1, 3]])
B_2x2 = np.array([5, 10])
# Ejecución
soluciones = resolver_2x2_paso_a_paso(A_2x2, B_2x2)
print("\n" + "="*30)
print("--- SOLUCIONES DIRECTAS ---")
print(f"x = {soluciones[0]}")
print(f"y = {soluciones[1]}")
print("="*30)
```

```
--- PASO 1: Matriz Aumentada Inicial ---
[[ 2.  1.  5.]
 [ 1.  3. 10.]]
PASO 3: Fila 1 normalizada (Pivote = 1):
[[ 1.   0.5  2.5]
 [ 1.   3.  10. ]]
PASO 4: Eliminando elemento en [1,0] usando factor1.0:
[[1.  0.5 2.5]
 [0.  2.5 7.5]]
PASO 3: Fila 2 normalizada (Pivote = 1):
[[1.  0.5 2.5]
 [0.  1.  3. ]]
PASO 4: Eliminando elemento en [0,1] usando factor0.5:
[[1. 0. 1.]
 [0. 1. 3.]]

==============================
--- SOLUCIONES DIRECTAS ---
x = 1.0
y = 3.0
==============================
```

Ejercicio 2

Matriz de 3x3

Sistema de Ecuaciones 3x3:

1. $3x + 2y - z = 1$
2. $2x - 2y + 4z = -2$
3. $-x + \frac{1}{2}y - z = 0$

```python
import numpy as np
def resolver_3x3_paso_a_paso(A, B):
    # PASO 1: Matriz Aumentada
    Ab = np.hstack([A.astype(float), B.reshape(-1, 1).astype(float)])
    n = len(B)
    print(Ab)
    for i in range(n):
        print(f"\n=== PROCESANDO COLUMNA {i+1} ===")
        # PASO 2: Pivoteo
        if abs(Ab[i, i]) < 1e-10:
            for k in range(i + 1, n):
                if abs(Ab[k, i]) > 1e-10:
                    Ab[[i, k]] = Ab[[k, i]]
                    print(f"PASO 2: Intercambio de fila {i+1} con fila {k+1}")
                    break
        # PASO 3: Normalización
        pivote = Ab[i, i]
        Ab[i] = Ab[i] / pivote
        print(f"PASO 3: Fila {i+1} normalizada (Pivote = 1):")
        print(Ab.round(2)) # Redondeo a 2 decimales para lectura limpia
        # PASO 4: Eliminación
        for j in range(n):
            if i != j:
                factor = Ab[j, i]
                Ab[j] = Ab[j] - factor * Ab[i]
                print(f"PASO 4: Eliminando elemento en [{j},{i}] (Factor: {factor:
        print(f"Estado tras Columna {i+1}:")
        print(Ab.round(2))
    # PASO 5: Repetición (Controlada por el ciclo 'for i')
    return Ab[:, -1]
# Datos de la matriz 3x3
A_3x3 = np.array([[3, 2, -1],
                  [2, -2, 4],
                  [-1, 0.5, -1]])
B_3x3 = np.array([1, -2, 0])
```

```python
# Ejecución
soluciones = resolver_3x3_paso_a_paso(A_3x3, B_3x3)
print("\n" + "="*35)
print("--- SOLUCIONES DIRECTAS (3x3) ---")
print(f"x = {soluciones[0]:.2f}")
print(f"y = {soluciones[1]:.2f}")
print(f"z = {soluciones[2]:.2f}")
print("="*35)
```

```
[[ 3.   2.  -1.   1. ]
 [ 2.  -2.   4.  -2. ]
 [-1.   0.5 -1.   0. ]]

=== PROCESANDO COLUMNA 1 ===
PASO 3: Fila 1 normalizada (Pivote = 1):
[[ 1.    0.67 -0.33  0.33]
 [ 2.   -2.    4.   -2.  ]
 [-1.    0.5  -1.    0.  ]]
PASO 4: Eliminando elemento en [1,0] (Factor: 2.00)
PASO 4: Eliminando elemento en [2,0] (Factor: -1.00)
Estado tras Columna 1:
[[ 1.    0.67 -0.33  0.33]
 [ 0.   -3.33  4.67 -2.67]
 [ 0.    1.17 -1.33  0.33]]

=== PROCESANDO COLUMNA 2 ===
PASO 3: Fila 2 normalizada (Pivote = 1):
[[ 1.    0.67 -0.33  0.33]
 [-0.    1.   -1.4   0.8 ]
 [ 0.    1.17 -1.33  0.33]]
PASO 4: Eliminando elemento en [0,1] (Factor: 0.67)
PASO 4: Eliminando elemento en [2,1] (Factor: 1.17)
Estado tras Columna 2:
[[ 1.   0.   0.6 -0.2]
 [-0.   1.  -1.4  0.8]
 [ 0.   0.   0.3 -0.6]]

=== PROCESANDO COLUMNA 3 ===
PASO 3: Fila 3 normalizada (Pivote = 1):
[[ 1.   0.   0.6 -0.2]
 [-0.   1.  -1.4  0.8]
 [ 0.   0.   1.  -2. ]]
PASO 4: Eliminando elemento en [0,2] (Factor: 0.60)
PASO 4: Eliminando elemento en [1,2] (Factor: -1.40)
Estado tras Columna 3:
[[ 1.  0.  0.  1.]
 [ 0.  1.  0. -2.]
 [ 0.  0.  1. -2.]]

===================================
--- SOLUCIONES DIRECTAS (3x3) ---
x = 1.00
y = -2.00
z = -2.00
===================================
```

Ejercicio 3

Matriz 5x5

1-. Matriz aumentada

Se escribe el sistema como **matriz aumentada (A|B)**:

$$\left[\begin{array}{ccccc|c} 1 & 2 & 1 & 1 & 1 & 10 \\ 2 & 3 & 1 & 1 & 1 & 13 \\ 1 & 1 & 2 & 1 & 1 & 12 \\ 1 & 1 & 1 & 2 & 1 & 11 \\ 1 & 1 & 1 & 1 & 2 & 14 \end{array}\right]$$

2-.Pivoteo (Columna 1)

El pivote es **1**, por lo que no se necesita intercambio de filas.

3-. Normalización

El pivote ya es **1**, así que la fila queda igual.

4-. Eliminación (hacer ceros en columna 1)

Operaciones:

F2 = F2 – 2F1

F3 = F3 – F1

F4 = F4 – F1

F5 = F5 – F1

Resultado:

$$\left[\begin{array}{ccccc|c} 1 & 2 & 1 & 1 & 1 & 10 \\ 0 & -1 & -1 & -1 & -1 & -7 \\ 0 & -1 & 1 & 0 & 0 & 2 \\ 0 & -1 & 0 & 1 & 0 & 1 \\ 0 & -1 & 0 & 0 & 1 & 4 \end{array}\right]$$

5-. Pivoteo (Columna 2)

El pivote es -1.

6-. Normalización

Dividimos la fila 2 entre **-1**

F2 = F2 / -1

$$\left[\begin{array}{ccccc|c} 1 & 2 & 1 & 1 & 1 & 10 \\ 0 & 1 & 1 & 1 & 1 & 7 \\ 0 & -1 & 1 & 0 & 0 & 2 \\ 0 & -1 & 0 & 1 & 0 & 1 \\ 0 & -1 & 0 & 0 & 1 & 4 \end{array}\right]$$

7. Eliminación (columna 2)

Operaciones:

F1 = F1 − 2F2
F3 = F3 + F2
F4 = F4 + F2
F5 = F5 + F2

Resultado:

$$\left[\begin{array}{ccccc|c} 1 & 0 & -1 & -1 & -1 & -4 \\ 0 & 1 & 1 & 1 & 1 & 7 \\ 0 & 0 & 2 & 1 & 1 & 9 \\ 0 & 0 & 1 & 2 & 1 & 8 \\ 0 & 0 & 1 & 1 & 2 & 11 \end{array}\right]$$

8. Pivoteo (columna 3)

El pivote es 2.

9. Normalización

F3 = F3 / 2

$$\left[\begin{array}{ccccc|c} 1 & 0 & -1 & -1 & -1 & -4 \\ 0 & 1 & 1 & 1 & 1 & 7 \\ 0 & 0 & 1 & 0.5 & 0.5 & 4.5 \\ 0 & 0 & 1 & 2 & 1 & 8 \\ 0 & 0 & 1 & 1 & 2 & 11 \end{array}\right]$$

10. Eliminación (columna 3)

F1 = F1 + F3
F2 = F2 − F3
F4 = F4 − F3
F5 = F5 − F3

Resultado:

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0 & -0.5 & -0.5 & 0.5 \\ 0 & 1 & 0 & 0.5 & 0.5 & 2.5 \\ 0 & 0 & 1 & 0.5 & 0.5 & 4.5 \\ 0 & 0 & 0 & 1.5 & 0.5 & 3.5 \\ 0 & 0 & 0 & 0.5 & 1.5 & 6.5 \end{array}\right]$$

11. Pivoteo (columna 4)

El pivote es **1.5**.

12. Normalización

F4 = F4 / 1.5

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0 & -0.5 & -0.5 & 0.5 \\ 0 & 1 & 0 & 0.5 & 0.5 & 2.5 \\ 0 & 0 & 1 & 0.5 & 0.5 & 4.5 \\ 0 & 0 & 0 & 1 & 0.33 & 2.33 \\ 0 & 0 & 0 & 0.5 & 1.5 & 6.5 \end{array}\right]$$

13. Pivoteo (columna 5)

Normalizamos la fila 5 y eliminamos los demás elementos hasta obtener la identidad.

Resultado final:

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 2 \\ 0 & 0 & 1 & 0 & 0 & 3 \\ 0 & 0 & 0 & 1 & 0 & 2 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{array}\right]$$

Solución

x= 1

y= 2

z= 3

w= 2

v= 4

Codigo matriz 5x5

```
def gauss_jordan(A):
    n = len(A)

    for i in range(n):

        # PIVOTEO (si el pivote es 0 se intercambian filas)
        if A[i][i] == 0:
            for k in range(i+1, n):
                if A[k][i] != 0:
                    A[i], A[k] = A[k], A[i]
                    break

        pivote = A[i][i]

        # NORMALIZACIÓN (hacer el pivote = 1)
        for j in range(n+1):
            A[i][j] = A[i][j] / pivote

        # ELIMINACIÓN (hacer ceros arriba y abajo)
        for k in range(n):
            if k != i:
                factor = A[k][i]
                for j in range(n+1):
                    A[k][j] = A[k][j] - factor * A[i][j]

    return A


# MATRIZ AUMENTADA DEL EJERCICIO 5x5
A = [
[1, 2, 1, 1, 1, 10],
[2, 3, 1, 1, 1, 13],
[1, 1, 2, 1, 1, 12],
[1, 1, 1, 2, 1, 11],
[1, 1, 1, 1, 2, 14]
]

resultado = gauss_jordan(A)
```

```
print("Matriz reducida:")
for fila in resultado:
    print(fila)

print("\nSolución del sistema:")

variables = ["x","y","z","w","v"]

for i in range(len(resultado)):
    print(variables[i], "=", resultado[i][-1])
```

Salida

```
Matriz reducida:
[1.0, 0.0, 0.0, 0.0, 0.0, 3.0]
[-0.0, 1.0, 0.0, 0.0, 0.0, -2.220446049250313e-16]
[0.0, 0.0, 1.0, 0.0, 0.0, 1.9999999999999996]
[0.0, 0.0, 0.0, 1.0, 0.0, 1.0000000000000002]
[0.0, 0.0, 0.0, 0.0, 1.0, 4.0]

Solución del sistema:
x = 3.0
y = -2.220446049250313e-16
z = 1.9999999999999996
w = 1.0000000000000002
v = 4.0
```

Ejercicio 4

Sistema de Ecuaciones 4x4:

1. $x + y + z + w = 10$
2. $2x + 3y + z + 5w = 31$
3. $-x + y - z + w = -2$
4. $3x + y + 2z + 4w = 28$

```python
# PASO 1: Definir matrices (Matriz de coeficientes A y resultados B)
A = np.array([
    [1, 1, 1, 1],
    [2, 3, 1, 5],
    [-1, 1, -1, 1],
    [3, 1, 2, 4]
], dtype=float)
B = np.array([10, 31, -2, 28], dtype=float)
def resolver_gauss_jordan_4x4(A, B):
    # Combinar en Matriz Aumentada
    Ab = np.hstack([A, B.reshape(-1, 1)])
    n = len(B)
    print("--- MATRIZ AUMENTADA INICIAL 4x4 ---")
    print(Ab, "\n")
    for i in range(n):
        # PASO 2: Pivoteo (asegurar que el pivote no sea cero)
        if abs(Ab[i, i]) < 1e-10:
            for k in range(i + 1, n):
                if abs(Ab[k, i]) > 1e-10:
                    Ab[[i, k]] = Ab[[k, i]]
                    break
        # PASO 3: Normalización (Hacer que el pivote sea 1)
        pivote = Ab[i, i]
        Ab[i] = Ab[i] / pivote
        # PASO 4: Eliminación (Ceros arriba y abajo en la columna i)
        for j in range(n):
            if i != j:
                factor = Ab[j, i]
                Ab[j] = Ab[j] - factor * Ab[i]
        # PASO 5: Repetición (Visualización por cada columna procesada)
        print(f"Columna {i+1} procesada:")
        print(Ab.round(2), "\n")
    return Ab[:, -1]
# Ejecución
soluciones = resolver_gauss_jordan_4x4(A, B)

print("--- SOLUCIONES DIRECTAS ---")
variables = ['x', 'y', 'z', 'w']
for var, val in zip(variables, soluciones):
    print(f"{var} = {round(val, 2)}")
```

```
--- MATRIZ AUMENTADA INICIAL 4x4 ---
[[ 1.  1.  1.  1. 10.]
 [ 2.  3.  1.  5. 31.]
 [-1.  1. -1.  1. -2.]
 [ 3.  1.  2.  4. 28.]]

Columna 1 procesada:
[[ 1.  1.  1.  1. 10.]
 [ 0.  1. -1.  3. 11.]
 [ 0.  2.  0.  2.  8.]
 [ 0. -2. -1.  1. -2.]]

Columna 2 procesada:
[[  1.   0.   2.  -2.  -1.]
 [  0.   1.  -1.   3.  11.]
 [  0.   0.   2.  -4. -14.]
 [  0.   0.  -3.   7.  20.]]

Columna 3 procesada:
[[ 1.  0.  0.  2. 13.]
 [ 0.  1.  0.  1.  4.]
 [ 0.  0.  1. -2. -7.]
 [ 0.  0.  0.  1. -1.]]

Columna 4 procesada:
[[ 1.  0.  0.  0. 15.]
 [ 0.  1.  0.  0.  5.]
 [ 0.  0.  1.  0. -9.]
 [ 0.  0.  0.  1. -1.]]

--- SOLUCIONES DIRECTAS ---
x = 15.0
y = 5.0
z = -9.0
w = -1.0
```

Ejercicio 5

Matriz de 6x6
La Matriz aumentada:

$$\left[\begin{array}{cccccc|c} 3 & 6 & 3 & 4 & 1 & 6 & 3 \\ 3 & 7 & 4 & 5 & 3 & 5 & 9 \\ 7 & 4 & 3 & 3 & 6 & 9 & 1 \\ 2 & 7 & 5 & 8 & 8 & 7 & 5 \\ 6 & 8 & 8 & 3 & 1 & 7 & 6 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

Código:

```python
import time
import copy

def gauss_jordan(matriz):
    """Resuelve un sistema de ecuaciones usando el método de Gauss-Jordan."""
    n = len(matriz)
    m = [fila[:] for fila in matriz]  # Copia profunda

    print("\n" + "="*60)
    print("       MÉTODO DE GAUSS-JORDAN - RESOLUCIÓN PASO A PASO")
    print("="*60)

    def imprimir_matriz(m, titulo=""):
        if titulo:
            print(f"\n  [{titulo}]")
        for fila in m:
            coefs = "  ".join(f"{x:10.4f}" for x in fila[:-1])
            print(f"  [ {coefs}  |  {fila[-1]:10.4f} ]")

    imprimir_matriz(m, "Matriz inicial aumentada")

    for col in range(n):
        # Buscar pivote máximo (pivoteo parcial)
        max_fila = col
        for fila in range(col + 1, n):
            if abs(m[fila][col]) > abs(m[max_fila][col]):
                max_fila = fila

        # Intercambiar filas si es necesario
        if max_fila != col:
            m[col], m[max_fila] = m[max_fila], m[col]
            print(f"\n  ↕ Intercambio: Fila {col+1} ⇔ Fila {max_fila+1}")
```

```
print("\n" + "="*60)
if soluciones:
    print("                    SOLUCIONES")
    print("="*60)
    for i, val in enumerate(soluciones, 1):
        print(f"  x{i} = {val:>12.6f}")

    print("\n" + "-"*60)
    print("  Verificación (sustituyendo en el sistema original):")
    print("-"*60)
    A = [fila[:-1] for fila in matriz]
    b = [fila[-1] for fila in matriz]
    ok = True
    for i in range(len(b)):
        resultado = sum(A[i][j] * soluciones[j] for j in range(len(soluciones)))
        estado = "✓" if abs(resultado - b[i]) < 1e-6 else "✗"
        print(f"  Ecuación {i+1}: {resultado:.6f}  (esperado {b[i]})  {estado}")
        if abs(resultado - b[i]) >= 1e-6:
            ok = False
    print("-"*60)
    print(f"  {'✅ Verificación exitosa' if ok else '⚠ Verifique los resultados'}")
else:
    print("  ⚠ El sistema no tiene solución única.")

print("\n" + "="*60)
print(f"  ⏱  Tiempo de ejecución: {tiempo_total:.6f} segundos")
print(f"     ({tiempo_total * 1_000:.4f} milisegundos)")
print("="*60 + "\n")
```

Resultados del codigo:

```
============================================================
                      SOLUCIONES
============================================================
  x1 =     1.073770
  x2 =     3.457377
  x3 =    -1.195082
  x4 =    -1.170492
  x5 =     1.288525
  x6 =    -2.331148

------------------------------------------------------------
  Verificación (sustituyendo en el sistema original):
------------------------------------------------------------
  Ecuación 1: 3.000000  (esperado 3)  ✓
  Ecuación 2: 9.000000  (esperado 9)  ✓
  Ecuación 3: 1.000000  (esperado 1)  ✓
  Ecuación 4: 5.000000  (esperado 5)  ✓
  Ecuación 5: 6.000000  (esperado 6)  ✓
  Ecuación 6: 6.000000  (esperado 6)  ✓
------------------------------------------------------------
```

Operaciones:

1.-

$$\text{Row operation : } R_1 \to \frac{R_1}{3}$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 3 & 7 & 4 & 5 & 3 & 5 & 9 \\ 7 & 4 & 3 & 3 & 6 & 9 & 1 \\ 2 & 7 & 5 & 8 & 8 & 7 & 5 \\ 6 & 8 & 8 & 3 & 1 & 7 & 6 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

2.-

$$R_2 \to R_2 - (3)R_1$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 7 & 4 & 3 & 3 & 6 & 9 & 1 \\ 2 & 7 & 5 & 8 & 8 & 7 & 5 \\ 6 & 8 & 8 & 3 & 1 & 7 & 6 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

3.-

$$R_3 \to R_3 - (7)R_1$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & -10 & -4 & -\frac{19}{3} & \frac{11}{3} & -5 & -6 \\ 2 & 7 & 5 & 8 & 8 & 7 & 5 \\ 6 & 8 & 8 & 3 & 1 & 7 & 6 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

4.-

$$R_4 \to R_4 - (2)R_1$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & -10 & -4 & -\frac{19}{3} & \frac{11}{3} & -5 & -6 \\ 0 & 3 & 3 & \frac{16}{3} & \frac{22}{3} & 3 & 3 \\ 6 & 8 & 8 & 3 & 1 & 7 & 6 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

5.-

$$R_5 \to R_5 - (6)R_1$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & -10 & -4 & -\frac{19}{3} & \frac{11}{3} & -5 & -6 \\ 0 & 3 & 3 & \frac{16}{3} & \frac{22}{3} & 3 & 3 \\ 0 & -4 & 2 & -5 & -1 & -5 & 0 \\ 3 & 5 & 2 & 7 & 6 & 5 & 6 \end{array}\right]$$

6.-

$$R_6 \to R_6 - (3)R_1$$

$$\left[\begin{array}{cccccc|c} 1 & 2 & 1 & \frac{4}{3} & \frac{1}{3} & 2 & 1 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & -10 & -4 & -\frac{19}{3} & \frac{11}{3} & -5 & -6 \\ 0 & 3 & 3 & \frac{16}{3} & \frac{22}{3} & 3 & 3 \\ 0 & -4 & 2 & -5 & -1 & -5 & 0 \\ 0 & -1 & -1 & 3 & 5 & -1 & 3 \end{array}\right]$$

7.-

$$R_1 \to R_1 - (2)R_2$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & -10 & -4 & -\frac{19}{3} & \frac{11}{3} & -5 & -6 \\ 0 & 3 & 3 & \frac{16}{3} & \frac{22}{3} & 3 & 3 \\ 0 & -4 & 2 & -5 & -1 & -5 & 0 \\ 0 & -1 & -1 & 3 & 5 & -1 & 3 \end{array}\right]$$

8.-

$$R_3 \to R_3 - (-10)R_2$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 6 & \frac{11}{3} & \frac{71}{3} & -15 & 54 \\ 0 & 3 & 3 & \frac{16}{3} & \frac{22}{3} & 3 & 3 \\ 0 & -4 & 2 & -5 & -1 & -5 & 0 \\ 0 & -1 & -1 & 3 & 5 & -1 & 3 \end{array}\right]$$

9.-

10.-

$R_4 \to R_4 - (3)R_2$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 6 & \frac{11}{3} & \frac{71}{3} & -15 & 54 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & -4 & 2 & -5 & -1 & -5 & 0 \\ 0 & -1 & -1 & 3 & 5 & -1 & 3 \end{array}\right]$$

$R_5 \to R_5 - (-4)R_2$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 6 & \frac{11}{3} & \frac{71}{3} & -15 & 54 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 6 & -1 & 7 & -9 & 24 \\ 0 & -1 & -1 & 3 & 5 & -1 & 3 \end{array}\right]$$

11.-

$R_6 \to R_6 - (-1)R_2$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 6 & \frac{11}{3} & \frac{71}{3} & -15 & 54 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 6 & -1 & 7 & -9 & 24 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

12.-

Row operation : $R_3 \to \frac{R_3}{6}$

$$\left[\begin{array}{cccccc|c} 1 & 0 & -1 & -\frac{2}{3} & -\frac{11}{3} & 4 & -11 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 6 & -1 & 7 & -9 & 24 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

13.-

$R_1 \to R_1 - (-1)R_3$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & -\frac{1}{18} & \frac{5}{18} & \frac{3}{2} & -2 \\ 0 & 1 & 1 & 1 & 2 & -1 & 6 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 6 & -1 & 7 & -9 & 24 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

14.-

$R_2 \to R_2 - (1)R_3$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & -\frac{1}{18} & \frac{5}{18} & \frac{3}{2} & -2 \\ 0 & 1 & 0 & \frac{7}{18} & -\frac{35}{18} & \frac{3}{2} & -3 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 6 & -1 & 7 & -9 & 24 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

15.-

$R_5 \to R_5 - (6)R_3$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & -\frac{1}{18} & \frac{5}{18} & \frac{3}{2} & -2 \\ 0 & 1 & 0 & \frac{7}{18} & -\frac{35}{18} & \frac{3}{2} & -3 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & \frac{7}{3} & \frac{4}{3} & 6 & -15 \\ 0 & 0 & 0 & -\frac{14}{3} & -\frac{50}{3} & 6 & -30 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

16.-

Row operation : $R_4 \to \frac{R_4}{\frac{7}{3}}$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & -\frac{1}{18} & \frac{5}{18} & \frac{3}{2} & -2 \\ 0 & 1 & 0 & \frac{7}{18} & -\frac{35}{18} & \frac{3}{2} & -3 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & 1 & \frac{4}{7} & \frac{18}{7} & -\frac{45}{7} \\ 0 & 0 & 0 & -\frac{14}{3} & -\frac{50}{3} & 6 & -30 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

17.-

18.-

$$R_1 \to R_1 - (-\frac{1}{18})R_4$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & 0 & \frac{13}{42} & \frac{23}{14} & -\frac{33}{14} \\ 0 & 1 & 0 & \frac{7}{18} & -\frac{35}{18} & \frac{3}{2} & -3 \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & 1 & \frac{4}{7} & \frac{18}{7} & -\frac{45}{7} \\ 0 & 0 & 0 & -\frac{14}{3} & -\frac{50}{3} & 6 & -30 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

$$R_2 \to R_2 - (\frac{7}{18})R_4$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & 0 & \frac{13}{42} & \frac{23}{14} & -\frac{33}{14} \\ 0 & 1 & 0 & 0 & -\frac{13}{6} & \frac{1}{2} & -\frac{1}{2} \\ 0 & 0 & 1 & \frac{11}{18} & \frac{71}{18} & -\frac{5}{2} & 9 \\ 0 & 0 & 0 & 1 & \frac{4}{7} & \frac{18}{7} & -\frac{45}{7} \\ 0 & 0 & 0 & -\frac{14}{3} & -\frac{50}{3} & 6 & -30 \\ 0 & 0 & 0 & 4 & 7 & -2 & 9 \end{array}\right]$$

19.-

$$R_3 \to R_3 - (\frac{27}{49})R_6$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & \frac{131}{122} \\ 0 & 1 & 0 & 0 & 0 & 0 & \frac{2109}{610} \\ 0 & 0 & 1 & 0 & 0 & 0 & -\frac{729}{610} \\ 0 & 0 & 0 & 1 & 0 & \frac{162}{49} & -\frac{435}{49} \\ 0 & 0 & 0 & 0 & 1 & -\frac{9}{7} & \frac{30}{7} \\ 0 & 0 & 0 & 0 & 0 & 1 & -\frac{711}{305} \end{array}\right]$$

20.-

$$R_4 \to R_4 - (\frac{162}{49})R_6$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & \frac{131}{122} \\ 0 & 1 & 0 & 0 & 0 & 0 & \frac{2109}{610} \\ 0 & 0 & 1 & 0 & 0 & 0 & -\frac{729}{610} \\ 0 & 0 & 0 & 1 & 0 & 0 & -\frac{357}{305} \\ 0 & 0 & 0 & 0 & 1 & -\frac{9}{7} & \frac{30}{7} \\ 0 & 0 & 0 & 0 & 0 & 1 & -\frac{711}{305} \end{array}\right]$$

21.-

$$R_5 \to R_5 - (-\frac{9}{7})R_6$$

$$\left[\begin{array}{cccccc|c} 1 & 0 & 0 & 0 & 0 & 0 & \frac{131}{122} \\ 0 & 1 & 0 & 0 & 0 & 0 & \frac{2109}{610} \\ 0 & 0 & 1 & 0 & 0 & 0 & -\frac{729}{610} \\ 0 & 0 & 0 & 1 & 0 & 0 & -\frac{357}{305} \\ 0 & 0 & 0 & 0 & 1 & 0 & \frac{393}{305} \\ 0 & 0 & 0 & 0 & 0 & 1 & -\frac{711}{305} \end{array}\right]$$

Resultados

$$x_1 = \frac{131}{122}$$
$$x_2 = \frac{2109}{610}$$
$$x_3 = -\frac{729}{610}$$
$$x_4 = -\frac{357}{305}$$
$$x_5 = \frac{393}{305}$$
$$x_6 = -\frac{711}{305}$$

Ejercicio 6

Matriz de **10x10**

```python
import numpy as np

# Configuración para que la consola muestre todas las columnas de la matriz 10x10
np.set_printoptions(precision=2, suppress=True, linewidth=200)

def resolver_gauss_jordan_10x10():
    n = 10

    # --- PASO 1: Creación de la Matriz Aumentada ---
    # Matriz A: Diagonal con 10s, resto con 1s
    A = np.ones((n, n)) + np.diag([9]*n)
    # Vector B: Resultados del 10 al 100
    B = np.arange(10, 110, 10).astype(float)

    # Unir para formar la matriz aumentada (10 filas x 11 columnas)
    Ab = np.hstack([A, B.reshape(-1, 1)])

    print("ESTADO INICIAL: MATRIZ AUMENTADA (10x11)")
    print(Ab)
    print("-" * 100)

    # --- INICIO DEL ALGORITMO ---
    for i in range(n):
        # PASO 2: Pivoteo (Intercambio de filas si es necesario)
        fila_max = np.argmax(np.abs(Ab[i:, i])) + i
        Ab[[i, fila_max]] = Ab[[fila_max, i]]

        # PASO 3: Normalización (Hacer que el pivote sea 1)
        pivote = Ab[i, i]
        Ab[i] = Ab[i] / pivote

        # PASO 4: Eliminación (Hacer ceros arriba y abajo del pivote)
        for j in range(n):
            if i != j:
                factor = Ab[j, i]
                Ab[j] = Ab[j] - factor * Ab[i]
```

```
        # Mostramos un estado intermedio en la iteración 5
        if i == 4:
            print(f"ESTADO INTERMEDIO: COLUMNA {i+1} PROCESADA")
            print(Ab)
            print("-" * 100)

    # --- RESULTADO FINAL ---
    print("ESTADO FINAL: MATRIZ EN FORMA ESCALONADA REDUCIDA (IDENTIDAD)")
    print(Ab)

    return Ab[:, -1]

# Ejecución del programa
try:
    soluciones = resolver_gauss_jordan_10x10()

    print("\n" + "="*40)
    print("RESULTADOS FINALES (Valores de x1 a x10)")
    print("="*40)
    for idx, sol in enumerate(soluciones):
        print(f"x{idx+1:02d} = {sol:8.4f}")
    print("="*40)

except Exception as e:
    print(f"Error durante la ejecución: {e}")
```

```
 [  1.   1.   1.   1.   1.   1.   1.   1.  10.   1.  90.]
 [  1.   1.   1.   1.   1.   1.   1.   1.   1.  10. 100.]]
----------------------------------------------------------------------------------------------------
ESTADO INTERMEDIO: COLUMNA 5 PROCESADA
[[ 1.    0.    0.    0.    0.    0.07  0.07  0.07  0.07  0.07 -0.08]
 [ 0.    1.    0.    0.    0.    0.07  0.07  0.07  0.07  0.07  1.03]
 [ 0.    0.    1.    0.    0.    0.07  0.07  0.07  0.07  0.07  2.14]
 [ 0.    0.    0.    1.    0.    0.07  0.07  0.07  0.07  0.07  3.25]
 [ 0.    0.    0.    0.    1.    0.07  0.07  0.07  0.07  0.07  4.37]
 [ 0.    0.    0.    0.    0.    9.64  0.64  0.64  0.64  0.64 49.29]
 [ 0.    0.    0.    0.    0.    0.64  9.64  0.64  0.64  0.64 59.29]
 [ 0.    0.    0.    0.    0.    0.64  0.64  9.64  0.64  0.64 69.29]
 [ 0.    0.    0.    0.    0.    0.64  0.64  0.64  9.64  0.64 79.29]
 [ 0.    0.    0.    0.    0.    0.64  0.64  0.64  0.64  9.64 89.29]]
----------------------------------------------------------------------------------------------------
ESTADO FINAL: MATRIZ EN FORMA ESCALONADA REDUCIDA (IDENTIDAD)
[[ 1.    0.    0.    0.    0.    0.    0.    0.    0.    0.   -2.11]
 [ 0.    1.    0.    0.    0.    0.    0.    0.    0.    0.   -0.99]
 [ 0.    0.    1.    0.    0.    0.    0.    0.    0.    0.    0.12]
 [ 0.    0.    0.    1.    0.    0.    0.    0.    0.    0.    1.23]
 [ 0.    0.    0.    0.    1.    0.    0.    0.    0.    0.    2.34]
 [ 0.    0.    0.    0.    0.    1.    0.    0.    0.    0.    3.45]
 [ 0.    0.    0.    0.    0.    0.    1.    0.    0.    0.    4.56]
 [ 0.    0.    0.    0.    0.    0.    0.    1.    0.    0.    5.67]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    1.    0.    6.78]
 [ 0.    0.    0.    0.    0.    0.    0.    0.    0.    1.    7.89]]

========================================
RESULTADOS FINALES (Valores de x1 a x10)
========================================
x01 =  -2.1053
x02 =  -0.9942
x03 =   0.1170
x04 =   1.2281
x05 =   2.3392
x06 =   3.4503
x07 =   4.5614
x08 =   5.6725
x09 =   6.7836
x10 =   7.8947
```

Método de Gauss-Seidel

Algoritmo

1: Plantear el sistema de ecuaciones lineales en forma matricial y establecer las condiciones iniciales del método

2: Normalizar cada fila dividiendo sus valores por el elemento de la diagonal principal.

3: Simplificar las ecuaciones para que el coeficiente de cada incógnita sea 1.

4: Realizar un primer cálculo rápido de las variables para tener un punto de partida.

5: Generar valores base que servirán como "borrador" para las mejoras sucesivas.

6: Iniciar un ciclo repetitivo para recalcular cada variable y ganar precisión y aplicar lambda para promediar el valor nuevo con el anterior.

7: Estabilizar la búsqueda del resultado evitando cambios bruscos en los números y validar la precisión.

Código

```
def Gseid(a, b, n, x, imax, es, lmbda):
    for i in range(n):
        dummy = a[i][i]
        for j in range(n):
            a[i][j] = a[i][j] / dummy
        b[i] = b[i] / dummy

    for i in range(n):
        suma = b[i]
        for j in range(n):
            if i != j:
                suma = suma - a[i][j] * x[j]
        x[i] = suma

    iter = 1
    while True:
        centinela = 1
        for i in range(n):
            old = x[i]
            suma = b[i]
            for j in range(n):
                if i != j:
                    suma = suma - a[i][j] * x[j]
```

```
            x[i] = lmbda * suma + (1.0 - lmbda) * old

            if centinela == 1 and x[i] != 0:
                ea = abs((x[i] - old) / x[i]) * 100
                if ea > es:
                    centinela = 0

        iter = iter + 1
        if centinela == 1 or iter >= imax:
            break

    return x, iter
```

Ejercicio 1

```
==============================
MATRIZ DEL SISTEMA 3x3
==============================
      x        1x        2x        3   |     B
------------------------------
ec1      12.350    1.036    5.401   |    -6.152
ec2      -7.947   18.398   -0.896   |    -9.457
ec3      -5.117    9.470   26.752   |    -8.622



======================================
RESULTADOS
======================================
x 1 = -0.374345
x 2 = -0.683145
x 3 = -0.152079

Iteraciones: 6
Tiempo de ejecución: 0.0001608000204965472 segundos
```

Ejercicio 2

```
========================================
MATRIZ DEL SISTEMA 4x4
========================================
      x        1x        2x        3x        4   |     B
----------------------------------------
ec1      15.989   -7.182   -1.724   -1.780   |    -9.973
ec2      -9.857   33.604   -0.091   -9.977   |     0.496
ec3       7.906    3.917   20.371    0.933   |    -6.747
ec4       1.154   -2.320    7.481   19.824   |     5.170



========================================
RESULTADOS
========================================
x 1 = -0.631040
x 2 = -0.075163
x 3 = -0.086581
x 4 = 0.321388

Iteraciones: 10
Tiempo de ejecución: 0.0001922000083141029 segundos
```

Ejercicio 3

```
================================================
MATRIZ DEL SISTEMA 5x5
================================================
      x        1x        2x        3x        4x        5   |     B
------------------------------------------------
ec1      27.280   -2.597    8.818    8.692    0.136   |    -8.965
ec2       8.327   36.291   -0.224    5.941   -9.358   |     3.285
ec3      -3.848    1.307   23.573    5.358   -3.476   |    -1.751
ec4       4.138    5.616   -8.938   32.832    3.606   |    -8.487
ec5      -0.501    1.306    6.884    5.780   23.818   |     1.450



====================================
RESULTADOS
====================================
x 1 = -0.203992
x 2 = 0.217533
x 3 = -0.034438
x 4 = -0.293196
x 5 = 0.125749

Iteraciones: 8
Tiempo de ejecución: 0.0002742999931797385 segundos
```

Ejercicio 4

MATRIZ DEL SISTEMA 15x15

Columnas x1 a x5

x	1x	2x	3x	4x	5
ec 1	73.646	5.530	-3.057	2.453	-8.268
ec 2	-4.688	70.400	-9.190	9.639	-4.016
ec 3	-6.261	1.637	64.489	3.635	1.016
ec 4	-6.880	2.668	5.048	73.694	0.286
ec 5	-9.559	4.902	7.565	-6.678	72.167
ec 6	0.605	-8.842	-5.693	2.035	0.707
ec 7	5.184	-0.429	1.916	1.084	-2.522
ec 8	-6.260	9.365	6.959	-2.461	-0.174
ec 9	-2.137	-8.777	-6.344	1.752	-5.752
ec 10	4.865	9.462	-5.776	-1.171	1.144
ec 11	-5.542	4.377	0.070	-8.609	4.759
ec 12	0.196	0.434	4.582	-0.328	7.263
ec 13	7.219	9.612	-7.376	6.752	-5.371
ec 14	6.736	3.397	1.824	-5.588	-8.136
ec 15	0.779	8.297	-5.810	8.237	2.732

Columnas x6 a x10

x	6x	7x	8x	9x	10
ec 1	-3.087	-3.118	-8.358	3.402	1.072
ec 2	-5.790	-5.878	-9.778	-3.811	-3.488
ec 3	6.936	-0.485	9.448	0.930	-5.246
ec 4	-3.810	8.424	3.226	-4.391	-1.348
ec 5	-8.063	8.607	5.774	-0.402	-1.915
ec 6	86.049	4.849	-6.244	9.077	-6.083
ec 7	-8.752	74.544	3.749	-9.049	-8.965
ec 8	7.419	1.592	78.809	-1.587	-6.041
ec 9	9.112	-1.819	8.100	84.385	1.868
ec 10	6.524	-8.847	1.183	0.892	72.296
ec 11	5.573	-1.989	5.042	9.907	3.998
ec 12	-1.857	5.757	-9.409	9.533	-9.344
ec 13	1.098	-3.784	-4.831	3.921	0.058
ec 14	-5.891	-6.171	1.480	3.077	3.895
ec 15	-0.381	-4.531	4.373	-0.450	-5.053

```
Columnas x11 a x15
      x        11x        12x        13x        14x        15   |      B
-----------------------------------------------------------------------------
ec  1     -9.526     -2.486     -6.350     -2.147      1.881   |     -1.892
ec  2     -3.463     -0.111     -0.654     -0.253      3.220   |      1.578
ec  3     -4.495      6.058      3.341      4.342     -1.535   |      2.568
ec  4     -8.794      5.199      7.942      2.582     -6.120   |     -5.243
ec  5     -3.041     -1.273     -0.497      2.361      1.836   |      5.980
ec  6      2.671      5.593     -3.891     -8.952     -8.272   |     -8.991
ec  7     -2.522      5.837      8.014      1.719      6.469   |     -0.979
ec  8     -0.481      4.698     -9.866      7.941      8.652   |     -8.045
ec  9      8.161      3.457      5.309      4.793     -8.205   |     -8.899
ec 10     -3.915      1.695     -6.323      9.399     -0.471   |     -4.956
ec 11     78.723      4.626      1.581     -9.576      3.175   |     -2.491
ec 12      9.248     86.218     -6.139     -8.445      0.634   |     -1.018
ec 13     -0.093     -6.305     68.189     -2.995      1.916   |     -6.544
ec 14     -3.938     -1.871     -2.580     67.786      0.447   |     -3.030
ec 15     -5.992      6.270      5.236      4.235     71.597   |     -9.236


================ RESULTADOS ================

x 1 = -0.036806
x 2 = 0.015993
x 3 = 0.070599
x 4 = -0.083808
x 5 = 0.058724
x 6 = -0.115474
x 7 = -0.011311
x 8 = -0.100929
x 9 = -0.069519
x10 = -0.054393
x11 = -0.020024
x12 = -0.039414
x13 = -0.078549
x14 = -0.050731
x15 = -0.106242

Iteraciones: 8
Tiempo: 0.0003092000260949135 segundos
```

Ejercicio 5

```
============================
MATRIZ DEL SISTEMA (18x18)
============================

Columnas x1 a x6
      x         1x         2x         3x         4x         5x         6
-------------------------------------------------------------------------------------------
ec  1    117.734     8.913    -8.210    -5.963     9.745    -8.109
ec  2     -1.078    88.350     0.002    -3.061    -4.334     9.753
ec  3      5.432     2.169    81.432    -1.934     9.045    -7.875
ec  4     -8.576     0.927     9.177   102.102    -2.681     0.961
ec  5      3.718     7.909     2.904     0.575    90.263    -0.918
ec  6     -1.690     5.982     4.199     9.012    -3.948   119.536
ec  7     -6.861    -8.956    -0.270     9.414     2.877    -7.181
ec  8      7.581     0.173     9.809    -1.021    -3.269    -0.901
ec  9     -3.567     4.119     9.700    -9.627     6.853    -7.643
ec 10     -4.581    -0.195     0.353    -8.880     6.464    -6.551
ec 11      6.918    -6.595     8.664     7.894    -4.403    -2.372
ec 12      2.638     4.732     0.745    -2.037     6.417     3.596
ec 13     -4.308    -0.390     6.481    -5.619     5.510     0.518
ec 14     -8.525     1.888    -5.286    -4.705     4.653    -9.052
ec 15     -0.828     7.605     7.158    -0.564     4.443     1.848
ec 16     -8.494    -1.431     5.223    -9.299     6.277     6.988
ec 17     -6.300    -4.911     9.690     4.487    -0.074     3.328
ec 18     -8.047     2.368     2.555    -3.052    -7.007    -6.689


Columnas x7 a x12
      x         7x         8x         9x        10x        11x        12
-------------------------------------------------------------------------------------------
ec  1     -3.253     7.739    -3.395    -5.983     4.649    -9.625
ec  2     -7.895     5.451    -5.917    -9.225     7.163     4.911
ec  3     -0.510    -1.237    -5.969     4.312    -4.874    -3.941
ec  4      4.925    -3.689    -4.514     8.512    -9.825    -3.031
ec  5      6.734    -3.167    -9.981     1.369    -1.895     1.282
ec  6      8.856    -5.393    -1.229    -9.127    -3.566    -8.321
ec  7     99.583    -1.069     6.196     9.595     4.667    -5.093
ec  8      9.034    92.789     5.331    -2.934     6.063    -1.141
ec  9     -1.503    -3.684   103.660    -7.026    -3.169    -2.421
ec 10      4.634     6.750    -8.518    94.600    -7.128    -7.253
ec 11      3.264     2.158    -9.977     6.631   100.696    -1.130
ec 12      1.854     3.176     4.355    -6.953    -7.721    85.247
ec 13     -0.270    -8.870    -8.926    -5.558    -9.526    -8.241
ec 14     -6.616     3.963    -0.542     6.233     3.222     2.677
ec 15     -7.197    -7.510     1.073    -6.890    -8.480     7.042
ec 16      7.823     3.586    -5.238    -8.808    -6.654    -9.049
ec 17      5.601    -8.268     6.194    -4.771    -8.215     5.499
ec 18     -9.884     2.022     6.894    -6.891     1.006     8.143
```

```
Columnas x13 a x18
      x        13x        14x        15x        16x        17x        18   |        B
-------------------------------------------------------------------------------------------
ec  1     -6.984      2.547      5.477      9.765      4.251     -7.514   |      4.281
ec  2      4.456     -9.819      0.058      4.050      1.961     -0.859   |      7.854
ec  3      2.999     -4.550     -3.937     -2.925      4.591     -5.607   |      8.281
ec  4     -4.495     -8.232     -0.500      8.744     -8.576     -5.261   |     -5.956
ec  5     -6.747     -6.027     -9.539      4.870     -5.636     -8.020   |      2.631
ec  6     -5.931     -9.487     -8.478     -3.653      9.619      7.771   |      9.558
ec  7      6.405     -0.447      2.792     -6.301      5.633      7.285   |      9.849
ec  8      5.388      8.698      0.874     -9.502      0.002     -6.421   |     -9.876
ec  9     -0.999     -8.652      5.088     -7.418      9.267     -6.156   |     -3.634
ec 10      1.874      3.116      1.354      3.656      8.327     -7.579   |      8.373
ec 11      4.113     -3.501     -6.120     -7.310     -8.360     -5.596   |     -4.037
ec 12      5.459     -6.470     -7.190      6.510     -6.987     -0.863   |      7.499
ec 13    100.920     -7.014      3.845      9.515     -2.788      5.114   |      8.796
ec 14      1.893     94.541     -4.876     -9.375      5.858     -4.389   |      1.759
ec 15      5.839      2.529    100.889     -9.548      1.685      7.090   |     -9.424
ec 16      2.750      1.732     -7.463    110.408      6.567     -5.279   |     -6.392
ec 17      8.168     -0.830      1.975     -2.408    101.043      9.734   |     -6.501
ec 18     -2.399      7.588     -5.513      2.439      9.529     98.310   |     -6.054



==============================
RESULTADOS
==============================

x 1 = 0.077514
x 2 = 0.111716
x 3 = 0.083243
x 4 = -0.084738
x 5 = -0.016523
x 6 = 0.076900
x 7 = 0.127532
x 8 = -0.140587
x 9 = -0.033287
x10 = 0.097484
x11 = -0.071471
x12 = 0.065013
x13 = 0.083068
x14 = 0.029474
x15 = -0.121426
x16 = -0.070148
x17 = -0.084467
x18 = -0.035565

Iteraciones: 8
Tiempo de ejecución: 0.00039130006916821003 segundos
```

Ejercicio 6

```
==================================
MATRIZ DEL SISTEMA (20x20)
==================================

Columnas x1 a x5
      x         1x         2x         3x         4x         5
-------------------------------------------------------------------------------------------
ec  1     83.738    -0.494    -5.239    -3.412     1.774
ec  2      2.611    98.335     1.318    -2.354     4.705
ec  3      0.222     3.927    98.614     2.528    -5.177
ec  4      4.621    -4.879     7.394    87.246     3.862
ec  5      1.881     7.509    -2.248    -7.637    91.804
ec  6      6.695     4.518     4.915    -3.581     1.486
ec  7      3.351     8.112    -9.282     7.638    -7.613
ec  8      6.070    -1.883     8.672     9.051    -6.615
ec  9     -8.091    -8.908     6.069    -0.056     4.244
ec 10      7.094    -9.196    -8.650     5.722    -4.100
ec 11      2.605     2.412    -8.378     2.424     3.952
ec 12      1.173    -5.252     2.182     0.604    -0.015
ec 13     -4.168    -5.606    -2.023    -8.741     2.952
ec 14      3.760    -5.139     2.088    -0.925     5.931
ec 15     -8.975     2.059     4.953    -9.583    -1.742
ec 16      2.661     8.917     9.278     5.234     9.991
ec 17      6.660    -6.694    -8.563    -6.093     7.782
ec 18      4.174     2.539    -1.096     1.449     3.313
ec 19      0.066     2.679     7.862    -1.815     4.252
ec 20      7.791     4.093    -6.132     9.244    -2.130


Columnas x6 a x10
      x         6x         7x         8x         9x        10
-------------------------------------------------------------------------------------------
ec  1      0.535     7.469     0.829    -1.521     8.513
ec  2      4.720    -0.688     4.506     3.355     2.806
ec  3     -8.360    -5.138    -8.633    -3.277    -8.509
ec  4      5.969    -5.566     0.810     0.042    -0.224
ec  5      6.031    -4.889     0.732     5.774    -2.523
ec  6    102.415    -1.520     6.185    -4.492     5.767
ec  7      0.493   105.627     0.796     4.942    -9.874
ec  8     -8.705    -5.010   106.941     7.484     4.316
ec  9     -9.643    -4.983    -5.728   121.334     3.957
ec 10     -4.702     4.472     1.480    -0.720   112.211
ec 11      8.872     4.592    -4.002    -8.392    -6.725
ec 12      6.794     8.749     3.383     8.107    -3.859
ec 13      9.617    -7.894     5.720    -2.971     7.354
ec 14      6.717    -9.623     5.512     4.122    -9.136
ec 15      8.210     1.090    -9.564    -9.250    -6.710
ec 16     -9.527    -6.542     2.074    -2.252     8.391
ec 17     -0.486    -7.592    -5.659    -1.798    -9.600
ec 18      4.744     0.869    -0.395    -7.490     2.441
ec 19      9.971    -1.450    -5.206     9.104    -1.973
ec 20     -2.080    -2.625     2.504     8.119    -7.145
```

Columnas x11 a x15

	x	11x	12x	13x	14x	15
ec	1	-0.005	3.413	4.486	-9.082	9.726
ec	2	7.595	-7.774	2.351	5.595	7.000
ec	3	-2.576	-8.545	6.955	-9.723	0.186
ec	4	-0.110	7.048	3.764	-5.487	8.971
ec	5	-4.293	-1.240	-2.613	1.670	-2.460
ec	6	6.456	-3.387	-9.647	-8.721	4.828
ec	7	-4.576	5.945	-6.004	-6.537	-0.162
ec	8	-0.620	-6.967	1.035	-0.365	-1.626
ec	9	0.140	-7.927	-7.721	-2.441	-9.257
ec	10	2.413	4.773	-5.883	2.185	-9.667
ec	11	100.409	4.558	1.651	-6.824	-4.226
ec	12	-9.805	90.183	-0.925	-1.794	0.757
ec	13	-2.873	-9.955	126.469	-4.660	9.372
ec	14	-1.081	0.865	-0.313	89.280	-5.502
ec	15	0.165	-8.388	-8.454	-9.764	127.606
ec	16	-6.486	-4.889	-6.693	7.667	9.366
ec	17	-7.412	4.856	1.667	-0.299	7.224
ec	18	7.565	-2.973	1.673	2.530	2.024
ec	19	-4.076	-3.236	3.721	-7.325	-8.402
ec	20	-5.211	-4.675	6.181	2.191	8.785

Columnas x16 a x20

	x	16x	17x	18x	19x	20	B
ec	1	-2.306	-0.539	-0.259	7.564	-5.229	-5.870
ec	2	9.571	7.685	4.697	-6.785	5.520	1.793
ec	3	7.310	-0.376	-5.118	2.479	-3.093	0.792
ec	4	-6.490	2.360	0.456	-2.537	4.355	-8.425
ec	5	7.940	8.175	3.912	0.094	-6.345	-7.089
ec	6	7.217	-4.628	4.377	8.123	0.366	-9.420
ec	7	-7.301	-0.465	-0.209	-2.262	9.892	-6.819
ec	8	1.027	-5.931	4.883	-7.933	5.420	-5.007
ec	9	-8.946	5.102	8.358	-8.611	5.210	9.509
ec	10	-5.778	3.714	-7.678	8.435	-4.663	0.484
ec	11	-6.297	-5.464	-7.721	-2.352	-2.159	-4.289
ec	12	-3.242	-1.137	-5.739	6.028	-6.801	8.166
ec	13	5.200	9.757	5.477	7.515	4.812	-3.497
ec	14	2.026	-0.606	2.101	-6.300	-8.639	-4.816
ec	15	4.065	3.019	3.297	5.332	-8.709	1.183
ec	16	134.465	7.896	2.363	7.478	-7.648	-3.378
ec	17	3.250	110.891	2.482	-7.853	5.738	6.807
ec	18	-9.979	-5.001	91.589	-8.763	9.873	7.102
ec	19	6.586	8.799	-9.309	118.151	-8.706	-9.927
ec	20	-3.724	8.645	0.966	-2.869	108.432	-6.043

```
==============================
RESULTADOS
==============================

x 1 = -0.069837
x 2 = 0.030442
x 3 = 0.004575
x 4 = -0.100543
x 5 = -0.097442
x 6 = -0.083087
x 7 = -0.074213
x 8 = -0.053026
x 9 = 0.056535
x10 = 0.017399
x11 = -0.022829
x12 = 0.108834
x13 = -0.023418
x14 = -0.057008
x15 = -0.000123
x16 = -0.021996
x17 = 0.052534
x18 = 0.099597
x19 = -0.082602
x20 = -0.052118

Iteraciones: 10
Tiempo de ejecución: 0.00054240005556494 segundos
```

Método de Jacobi

Algoritmo

1.Escribir el sistema de ecuaciones lineales
2.Despejar cada variable en su respectiva ecuación
3.Elegir valores iniciales para las variables X1 = 0, x2 = 0, x3 = 0
4.Sustituir los valores iniciales en las ecuaciones despejadas para calcular los nuevos valores.
5.Obtener la primera iteración
6.Repetir el proceso usando los valores obtenidos para calcular la siguiente iteración
7.Calcular el error entre el valor nuevo y el anterior.
8.Comparar el error con la tolerancia establecida.
9.Detener el proceso cuando el error sea menor que la tolerancia.
10.Mostrar los valores finales como la solución aproximada del sistema.

Código

```
from numpy import*
A = array([[5.0, 2.0, -3.0],
           [2.0, 10.0, -8.0],
           [3.0, 8.0, 13.0]])

b = array([1.0, 4.0, 7.0]) # introducir matriz y vector independiente
n = len(A)

x0 = array([1.0, 1.0, 1.0])
x  = array([1.0, 1.0, 1.0])

nmax = 100
eps = 1e-10
k = 1

while (k <= nmax):
    for i in range(n):
        suma = 0.0
        for j in range(n):
            if j != i:
                suma = suma + A[i,j] * x0[j]
        x[i] = (b[i] - suma) / A[i,i]

    error = max(abs(x - x0))
```

```
    if error < eps:
        print("MÉTODO DE JACOBI")
        print("1) la solución es =", round(x[0],4), round(x[1],4),
round(x[2],4))
        print("2) itera =", k)
        print("3) error =", round(error,4))
        break

    for i in range(n):
        x0[i] = x[i]

    k = k + 1

if k >= nmax:
    print("no converge")
```

Ejercicio 1

```
from numpy import *
import time

A = array([[5.0, 2.0, -3.0],
           [2.0, 10.0, -8.0],
           [3.0, 8.0, 13.0]])
b = array([1.0, 4.0, 7.0])

n = len(A)
x0 = array([1.0, 1.0, 1.0])
x  = array([1.0, 1.0, 1.0])
nmax = 100
eps = 1e-10
k = 1

inicio = time.perfcounter()

while (k <= nmax):
    for i in range(n):
        suma = 0.0
        for j in range(n):
            if j != i:
                suma = suma + A[i,j] * x0[j]
```

```
            x[i] = (b[i] - suma) / A[i,i]

        error = max(abs(x - x0))

        if error < eps:
            fin = time.perf_counter()
            tiempo = fin - inicio
            print("MÉTODO DE JACOBI")
            print(" 1) La solución es =", [float(f"{v:.4g}") for v in x])
            print(" 2) Iteraciones =", k)
            print(" 3) Error =", float(f"{error:.4g}"))
            print(f" 4) Tiempo de ejecución = {tiempo:.4g} segundos")
            break

        for i in range(n):
            x0[i] = x[i]
        k = k + 1

    if k > nmax:
        print("No converge")
```

RESULTADO

```
MÉTODO DE JACOBI
 1) La solución es = [0.1009, 0.5307, 0.1886]
 2) Iteraciones = 95
 3) Error final = 8.161e-11
 4) Tiempo de ejecución = 0.009586 segundos
```

Ejercicio 2

Despeje del sistema de ecuaciones

$$x = \frac{9 - 2y - z}{10}$$

$$y = \frac{13 - 2x - 3z}{10}$$

$$z = \frac{14 - x - 3y}{10}$$

```
import numpy as np

A = np.array([[10,2,1],
              [2,10,3],
              [1,3,10]],float)

b = np.array([9,13,14],float)
```

```
x = np.zeros(3)

for k in range(10):

    x_new = np.zeros(3)

    x_new[0] = (9 -2*x[1] -x[2])/10
    x_new[1] = (13 -2*x[0] -3*x[2])/10
    x_new[2] = (14 -x[0] -3*x[1])/10

    x = x_new

print(x)
```

```
[0,62146316 0,8507832 1,082445 ]

=== Ejecución del código exitosa ===
```

Ejercicio 3

Sistema de ecuaciones

$$5x + y + z = 7$$

$$x + 4y + z = 6$$

$$x + y + 3z = 5$$

Despeje del sistema de ecuaciones

$$x = \frac{7 - y - z}{5}$$

$$y = \frac{6 - x - z}{4}$$

$$z = \frac{5 - x - y}{3}$$

```
import numpy as np

x = np.zeros(3)

for k in range(10):

    x_new = np.zeros(3)
```

```
    x_new[0] = (7 - x[1] - x[2]) / 5
    x_new[1] = (6 - x[0] - x[2]) / 4
    x_new[2] = (5 - x[0] - x[1]) / 3

    x = x_new

print("Solución aproximada:",x)
```

```
Solución aproximada: [0.9988963 0.99871037 0.99845086]

=== Ejecución del código exitosa ===
```

Ejercicio 4

Sistema de ecuaciones

$$3x + y + z = 5$$

$$x + 3y + z = 5$$

$$x + y + 3z = 5$$

Despeje del sistema de ecuaciones

$$x = \frac{5 - y - z}{3}$$

$$y = \frac{5 - x - z}{3}$$

$$z = \frac{5 - x - y}{3}$$

```
import numpy as np

x = np.zeros(3)

for k in range(25):

    x_new = np.zeros(3)

    x_new[0] = (5 - x[1] - x[2]) / 3
    x_new[1] = (5 - x[0] - x[2]) / 3
    x_new[2] = (5 - x[0] - x[1]) / 3
```

```
    x = x_new

print("Solución aproximada:",x)
```

Ejercicio 5

Sistema de ecuaciones

$$10x_1 + x_2 + x_3 + x_4 + x_5 + x_6 = 15$$

$$x_1 + 11x_2 + x_3 + x_4 + x_5 + x_6 = 16$$

$$x_1 + x_2 + 12x_3 + x_4 + x_5 + x_6 = 17$$

$$x_1 + x_2 + x_3 + 13x_4 + x_5 + x_6 = 18$$

$$x_1 + x_2 + x_3 + x_4 + 14x_5 + x_6 = 19$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + 15x_6 = 20$$

Despeje del sistema de ecuaciones

$$x_1 = \frac{15 - x_2 - x_3 - x_4 - x_5 - x_6}{10}$$

$$x_2 = \frac{16 - x_1 - x_3 - x_4 - x_5 - x_6}{11}$$

$$x_3 = \frac{17 - x_1 - x_2 - x_4 - x_5 - x_6}{12}$$

$$x_4 = \frac{18 - x_1 - x_2 - x_3 - x_5 - x_6}{13}$$

$$x_5 = \frac{19 - x_1 - x_2 - x_3 - x_4 - x_6}{14}$$

$$x_6 = \frac{20 - x_1 - x_2 - x_3 - x_4 - x_5}{15}$$

Código del ejercicio

```
import numpy as np

x = np.zeros(6)

for k in range(30):

    x_new = np.zeros(6)

    x_new[0] = (18 - x[1] - x[2] - x[3] - x[4] - x[5]) / 10
    x_new[1] = (20 - x[0] - x[2] - x[3] - x[4] - x[5]) / 11
    x_new[2] = (16 - x[0] - x[1] - x[3] - x[4] - x[5]) / 12
    x_new[3] = (19 - x[0] - x[1] - x[2] - x[4] - x[5]) / 13
    x_new[4] = (17 - x[0] - x[1] - x[2] - x[3] - x[5]) / 14
    x_new[5] = (21 - x[0] - x[1] - x[2] - x[3] - x[4]) / 15

    x = x_new

print("Solución aproximada:", x)
```

Salida

```
Solución aproximada: [1.2867258  1.35805322 0.87095747 1.04837768 0.81388709 1.04146658]
```

Ejercicio 6

Sistema de ecuaciones

$$16x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 = 20$$

$$x_1 + 17x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8 = 25$$

$$x_1 + x_2 + 18x_3 + x_4 + x_5 + x_6 + x_7 + x_8 = 30$$

$$x_1 + x_2 + x_3 + 19x_4 + x_5 + x_6 + x_7 + x_8 = 22$$

$$x_1 + x_2 + x_3 + x_4 + 20x_5 + x_6 + x_7 + x_8 = 27$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + 21x_6 + x_7 + x_8 = 24$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + 22x_7 + x_8 = 28$$

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + 23x_8 = 26$$

Despeje del sistema de ecuaciones

$$x_1 = \frac{20 - x_2 - x_3 - x_4 - x_5 - x_6 - x_7 - x_8}{16}$$

$$x_2 = \frac{25 - x_1 - x_3 - x_4 - x_5 - x_6 - x_7 - x_8}{17}$$

$$x_3 = \frac{30 - x_1 - x_2 - x_4 - x_5 - x_6 - x_7 - x_8}{18}$$

$$x_4 = \frac{22 - x_1 - x_2 - x_3 - x_5 - x_6 - x_7 - x_8}{19}$$

$$x_5 = \frac{27 - x_1 - x_2 - x_3 - x_4 - x_6 - x_7 - x_8}{20}$$

$$x_6 = \frac{24 - x_1 - x_2 - x_3 - x_4 - x_5 - x_7 - x_8}{21}$$

$$x_7 = \frac{28 - x_1 - x_2 - x_3 - x_4 - x_5 - x_6 - x_8}{22}$$

$$x_8 = \frac{26 - x_1 - x_2 - x_3 - x_4 - x_5 - x_6 - x_7}{23}$$

```python
import numpy as np

x = np.zeros(8)

for k in range(35):

    x_new = np.zeros(8)

    x_new[0] = (20 - x[1] - x[2] - x[3] - x[4] - x[5] - x[6] - x[7]) / 16
    x_new[1] = (25 - x[0] - x[2] - x[3] - x[4] - x[5] - x[6] - x[7]) / 17
    x_new[2] = (30 - x[0] - x[1] - x[3] - x[4] - x[5] - x[6] - x[7]) / 18
    x_new[3] = (22 - x[0] - x[1] - x[2] - x[4] - x[5] - x[6] - x[7]) / 19
    x_new[4] = (27 - x[0] - x[1] - x[2] - x[3] - x[5] - x[6] - x[7]) / 20
    x_new[5] = (24 - x[0] - x[1] - x[2] - x[3] - x[4] - x[6] - x[7]) / 21
    x_new[6] = (28 - x[0] - x[1] - x[2] - x[3] - x[4] - x[5] - x[7]) / 22
    x_new[7] = (26 - x[0] - x[1] - x[2] - x[3] - x[4] - x[5] - x[6]) / 23

    x = x_new

print("Solución aproximada:", x)
```

```
Solución aproximada: [0.82293053 1.08399737 1.31435047 0.79688655 1.01810305 0.8171979
 0.9687599  0.83381627]
```